IntelligentHome Project Report

Tianding Zhang, Yanlin Chen

December 18, 2024

Contents

1 Introduction **3**

2 Project Summary **3**

3 Project Summary and Implementation Details **4**

4 Objectives vs. Reality **8**

4.1 Proposed Objectives 8

4.2 Reality and Achievements 9

5 Post-Mortem and Conclusion **9**

5.1 What Went Right 9

5.2 What Went Wrong 10

6 Conclusion **11**

7 Appendix **12**

A UML Diagram **12**

1 Introduction

The *IntelligentHome* system is a prototype smart home controller designed to manage rooms, devices, and user access levels through an integrated, centralized platform. Implemented in Python and structured using a Model-View-Controller (MVC) pattern, the system enables the addition and configuration of rooms (Kitchen, Living Room, Bedroom), the control of various devices (Television, Lamp, Bedside Lamp, Refrigerator, Electric Kettle, Air Conditioner) and the maintenance of comfortable room temperatures using autonomous control logic. Users interact with the system through a graphical interface, where different access levels (MASTER, Admin, User) determine the range of allowed operations, such as adding rooms, adding or removing devices, or adjusting the state of two states or the heating and cooling levels of an air conditioner.

The key features of *IntelligentHome* include secure user authentication, persistent data storage using JSON files, and background temperature management. While only one room is visible at a time, all rooms continue to operate and update in real-time. The states of the devices, the heating / cooling actions and the temperature adjustments reflect realistic conditions, providing an environment where the user experience closely simulates a real-world intelligent home system.

2 Project Summary

This section highlights the primary capabilities and intended functionalities of the *IntelligentHome* system, offering an overview of how users, rooms, and devices are managed, as well as how data is stored and updated.

- **User Authentication and Access Control:** The system enforces a hierarchical access model using three roles: MASTER (highest), Admin, and User. MASTER accounts can register both Admin and User accounts, manage all rooms, and control all devices. Admin accounts can register User accounts and manage a subset of functions, while User accounts have limited, basic control (e.g., turning devices on/off). Login and registration logic ensures credentials are verified and that account creation strictly respects these access boundaries.

- **Room and Device Management:** Users with sufficient privileges can create rooms of designated types (Kitchen, Bedroom, Living Room) and populate them with allowed devices. Each device type has constraints on room compatibility and maximum counts. Interactive GUI elements allow adding devices, removing devices, and switching device states.

- **Device State Control and Temperature Dynamics:** Devices can be toggled on/off or adjusted through multiple states. Two-state devices (e.g., Lamps, TVs)

generate heat when switched on, while Air Conditioners support multiple heating and cooling levels. A central (conceptual) air conditioner automatically maintains room temperature within comfortable bounds, responding to environmental changes without direct user intervention.

- **Concurrent Room Temperature Management:** Multiple rooms run temperature calculations in parallel. Even if a room is not currently displayed, its devices and temperature remain continuously updated. Switching to a room's tab reveals its latest state, ensuring that the interface always reflects real-time conditions.
- **Persistent Data Storage:** Using JSON files (`users.json` and `tabs.json`), the system preserves user accounts, rooms, and devices. Upon startup, these files are loaded into memory as objects, reconstructing the previous session's configuration. While device states are reset to OFF at startup, the system restores the structural layout of rooms and assigned devices.
- **Temperature Control Logic:** Embedded control logic adjusts room temperature automatically. If the temperature dips below a predefined lower bound, the system heats the room to a target level; if it rises above an upper bound, it cools it down to the same target level. This finite-state-machine-based logic ensures stable, predictable environmental conditions.

Additionally, a UML diagram summarizing the system's design is included in Appendix A for reference.

3 Project Summary and Implementation Details

This section provides a closer look at the technical considerations, data structures, and implementation strategies used in *IntelligentHome*. Discuss how user authentication is validated, how rooms and devices are instantiated and managed, how device states influence temperature, and how concurrent execution ensures timely updates.

- **User Authentication and Access Control Implementation:** Authentication logic begins at startup by loading `users.json`, which stores credentials and access levels. The `AccessLevel` enumeration (MASTER=3, Admin=2, User=1) underpins permission checks. An example is showed in Fig. 1 For example, registering a new Admin requires the initiator to hold MASTER privileges, while adding a User requires at least Admin level. The controller verifies inputs (e.g., correct username/password, non-duplicate accounts, no empty entry), ensuring robust security. The GUI dynamically adjusts available buttons and operations depending on the current user's access level in both the login view and pop-up window.

```
{
    "1": {
        "password": "1",
        "access_level": 3
    },
    "2": {
        "password": "2",
        "access_level": 2
    },
    "3": {
        "password": "3",
        "access_level": 1
    }
}
```

Figure 1: User Credentials Stored in a JSON File Example

- **Room and Device Management Implementation:** Room and device data is retrieved from `tabs.json` at program start. The model instantiates tab objects (one per room) and device objects as defined in the file. Adding or removing rooms/devices involves updating these objects and saving changes back to `tabs.json`, see Fig. 2 for a example. The GUI, in sync with the controller, refreshes to reflect newly added rooms or devices and their states. For instance,

```
[
    {
        "tab_type": "Living Room",
        "tab_id": 1,
        "room": {
            "type": "Living Room",
            "id": 1,
            "devices": [
                {
                    "device_type": "Air Conditioner",
                    "device_id": 1
                }
            ]
        }
    },
    {
        "tab_type": "Kitchen",
        "tab_id": 1,
        "room": {
            "type": "Kitchen",
            "id": 1,
            "devices": [
                {
                    "device_type": "Refrigerator",
                    "device_id": 1
                }
            ]
        }
    },
    {
        "tab_type": "Bedroom",
        "tab_id": 1,
        "room": {
            "type": "Bedroom",
            "id": 1,
            "devices": [
                {
                    "device_type": "Bedside Lamp",
                    "device_id": 1
                }
            ]
        }
    }
]
```

Figure 2: Tab Credentials Stored in a JSON File Example

when a user with MASTER privileges clicks "Add Room," a dialog prompts for the room type. On confirmation, a new room object is created, saved, and displayed. Adding a device similarly updates internal lists and triggers GUI refresh. Removing a device locates it by device type, removes it from the internal structures from the highest ID value, updates the JSON file, and redraws the GUI. While the structural changes are saved, runtime states (e.g., ON/OFF) are not persisted, ensuring all devices default to OFF on restart.

- **Device State Control and Effects on Temperature:** Devices fall into three categories:

  1. *Two-State Devices (TV, Lamp, Electric Kettle, etc.):* These have ON/OFF states. Switching ON increases the room's heat input based on device-specific heating rates. Switching OFF removes that heat contribution. A finite state diagram is showed in Fig. 3

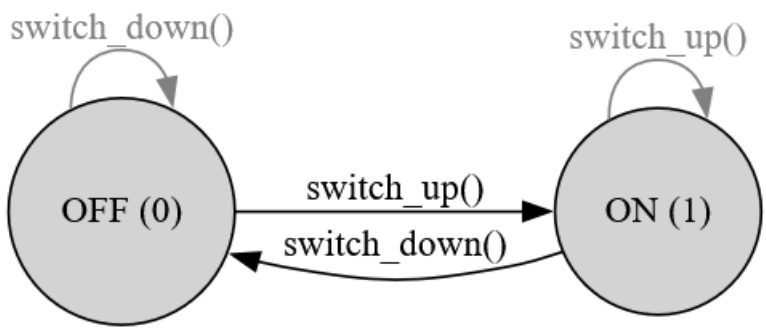


Figure 3: Two States Device Finite Element Machine Diagram

  2. *Multi-State Device (Air Conditioner):* This device supports 11 states: 5 cooling levels, 5 heating levels, and OFF. Increasing the cooling or heating level raises the magnitude of temperature change per calculation cycle. A finite state diagram is showed in Fig. 4

  3. *Central Air Conditioner (Control-Driven):* This is not an explicit device object but a conceptual control element within the room model. It engages heating or cooling automatically to maintain temperature within defined thresholds. Its logic references the finite state machine design to transition between OFF, COOLING, and HEATING states, adjusting room temperature accordingly. More details will be explored in the temperature control logic section.

State changes occur through controller methods that update device objects. While structural updates (adding/removing devices) prompt a JSON save, state changes like switching a lamp ON do not—these reflect ephemeral runtime changes. The main view updates device icons, labels, and indicators to mirror current states.

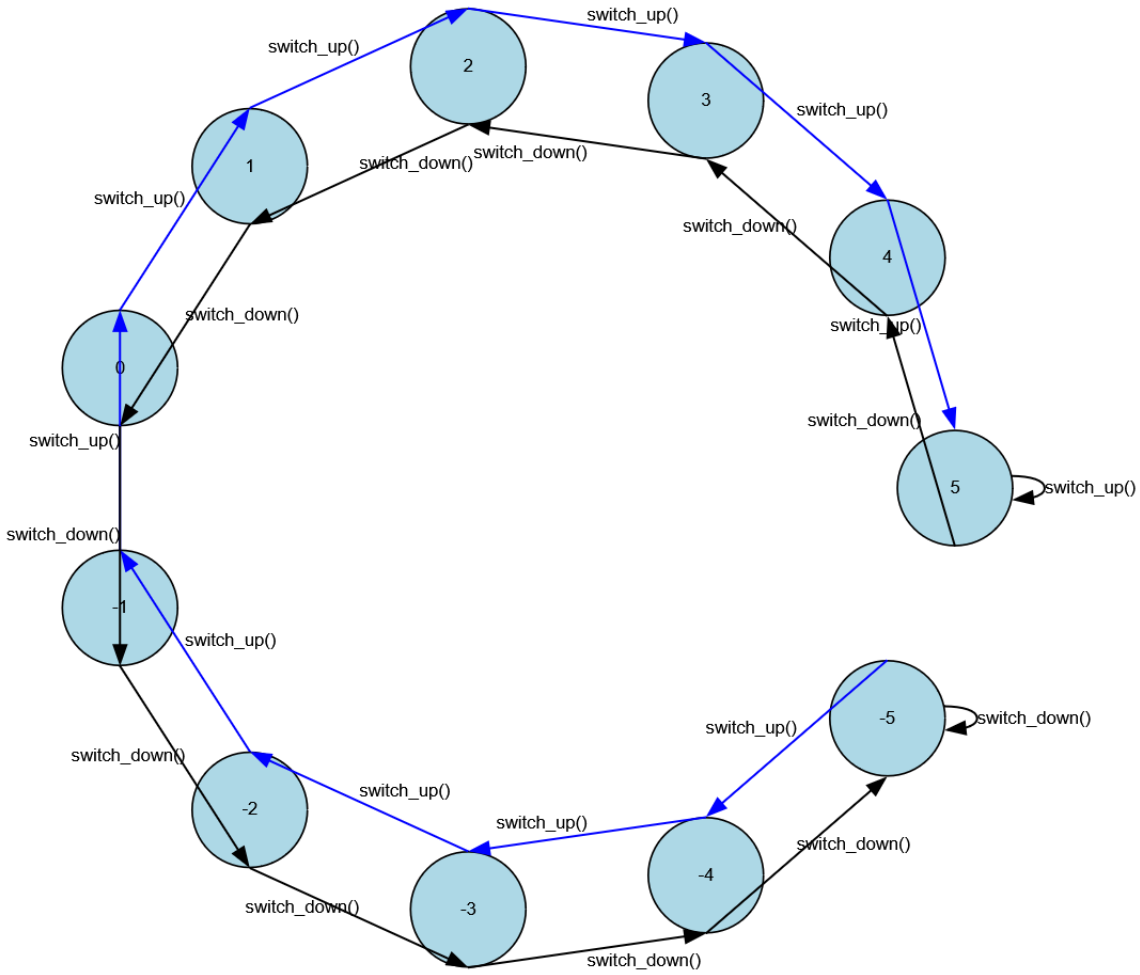


Figure 4: Air Conditioner Finite Element Machine Diagram

- **Multiple Room Temperature Management and Multithreading:** Real-time temperature management leverages concurrent execution. Each room's temperature updates run in a **dedicated thread**. The system uses a `calculation_refresh_rate` to periodically recompute temperatures based on active heating/cooling devices. Simultaneously, the GUI uses a `display_refresh_rate` to update visible information.

  This parallelism ensures that while the user may view and interact with only one room, all rooms are continuously managed. Switching between tabs instantaneously shows the latest states and temperatures of other rooms, eliminating stale or inconsistent data displays.

- **Persistent Data Storage Details:** The `users.json` file includes objects keyed by username, containing password and access level fields as showed in Fig. 1. On startup, the user model parses this file to build internal user objects. The `tabs.json` file , as showed in Fig. 2,contains a list of tabs, each describing a room's type and ID, plus a nested list of devices (with type and ID). The controller and model

coordinate reading and writing these files whenever structural changes occur (e.g., adding rooms, adding/removing devices).

By distinguishing between persistent room/device structures and transient runtime states (device ON/OFF, AC settings), the system ensures a known baseline at startup while still preserving user, room, and device arrangements from previous sessions.

- **Temperature Control Logic:** The control logic references predefined `lower_bound`, `upper_bound`, and `average` temperatures. If the room's current temperature falls below the lower bound, the central AC transitions to a HEATING state, raising the temperature back to the average before switching OFF. Conversely, if the room overheats above the upper bound, it enters a COOLING state until the average is restored. A finite state machine diagram, as showed in Fig. 5, guides these transitions, ensuring that triggers (e.g., temperature crossing thresholds) consistently produce correct, predictable behavior.

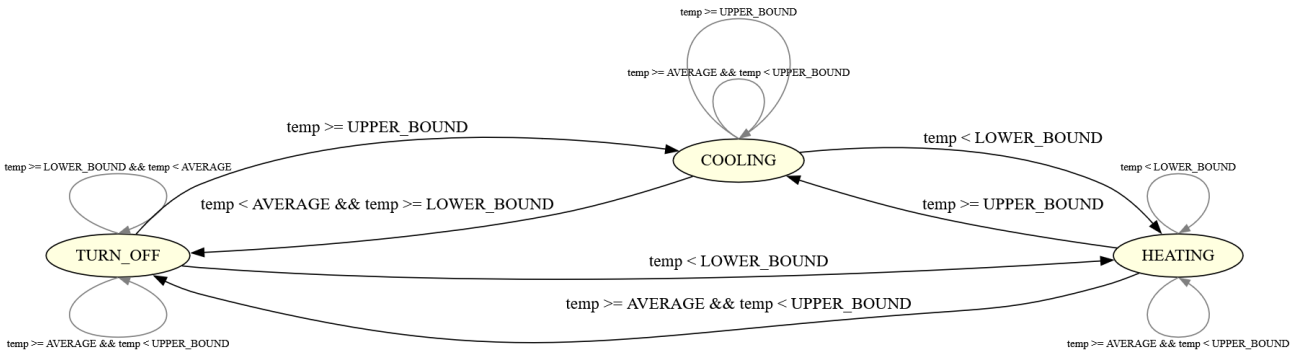


Figure 5: Center Air Conditioner Finite Element Machine Diagram

4 Objectives vs. Reality

4.1 Proposed Objectives

The proposed objectives for the Intelligent Home Appliance Control System, as outlined in the project proposal, are as follows:

- **User Authentication and Access Control:** Provide a secure login interface with username and password verification. Implement multi-level access control, where administrators can create and manage user accounts.

- **Room and Appliance Management:** Develop a centralized control panel to manage rooms and appliances. Users can add, remove, and monitor appliances within specific rooms (e.g., Living Room, Kitchen, Bedroom) with real-time status updates.

- **Environmental Monitoring and Automation:** Implement a time-based simulation to monitor room temperature every second. Automate safety protocols, such as controlling air conditioning, to maintain temperature thresholds.
- **Automated Safety Features:** Integrate automatic responses for environmental conditions, ensuring safe and comfortable room temperatures.

4.2 Reality and Achievements

- **User Authentication and Access Control:** The system provides a secure login interface with username and password verification. Multi-level access control is fully implemented:
  - *MASTER:* Registers Admin and User accounts, and manages all rooms, devices, and controls.
  - *Admin:* Registers User accounts and manages devices.
  - *User:* Adjusts device states, including turning devices ON/OFF and controlling air conditioner levels.
- **Room and Appliance Management:** The centralized control panel allows dynamic room and device management. Users can add rooms (Kitchen, Living Room, Bedroom) and populate them with devices such as Televisions, Lamps, and Air Conditioners, subject to room-specific constraints. Room tabs dynamically update to reflect real-time device and temperature data.
- **Environmental Monitoring and Automation:** Each room independently calculates real-time temperature changes based on device heat output and the center air conditioner logic. The system automatically maintains temperature within preset thresholds, activating heating or cooling as needed.
- **Automated Safety Features:** The center air conditioner operates with three states (Heating, Cooling, OFF) to ensure room temperatures stay within the desired range. This automatic response system simulates real-world climate control for user convenience and safety.

5 Post-Mortem and Conclusion

5.1 What Went Right

- **MVC Architecture:** The system adopted a clean Model-View-Controller structure. The model handled data logic (e.g., loading and saving from `users.json` and

`tabs.json`), the view dynamically updated the graphical interface, and the controller processed user inputs. This separation improved scalability, maintainability, and modularity.

- **Multithreading:** Background temperature calculations were performed using multithreading, ensuring smooth and non-blocking operations in the main graphical interface. This allowed real-time updates without lag.
- **Finite State Machine:** The center air conditioner logic employed a finite state machine to manage transitions between heating, cooling, and off states. This ensured all scenarios were considered and handled robustly.
- **Code Readability:** Enumerations were used for user access levels and device states. Mapping tables and lambda functions simplified logic for device control, user permissions, and state transitions, enhancing readability and reducing code complexity.
- **Object-Oriented Design:** The system adhered to OOP principles, representing rooms, devices, and users as distinct classes with clear responsibilities. This modular approach ensured reusable and maintainable code.

5.2 What Went Wrong

- **Simplistic GUI:** The interface lacks visual appeal, and refreshing content causes noticeable flickering, impacting the user experience.
- **Unmanaged Pop-Up Windows:** Multiple pop-up windows (e.g., Register, Add Device/Room) can be opened simultaneously, making the control flow messy.
- **Thread Management Issues:** Multithreading is not properly managed. While it works for now, as rooms operate independently and only one is displayed at a time, this limits scalability.
- **Control System Limitations:** The temperature control logic uses a linear heating rate and operates as an open-loop system, lacking feedback from sensors for real-world accuracy.
- **Poor Time Management:** Most features were implemented in the final week, reducing time for testing and refinement.
- **Incomplete Features:** Some functions remain unfinished, and the `Room` and `Device` classes lack generalization, reflecting time constraints and early design plans.

6 Conclusion

The IntelligentHome project successfully demonstrates a functional prototype for centralized home control, integrating user management, room and device operations, and temperature control within a modular MVC framework. Key achievements include the implementation of secure multi-level access control, dynamic room and device management, and real-time environmental simulation with automatic temperature regulation. JSON-based persistent storage ensures data continuity, while multithreading enables independent room temperature updates without blocking the main interface.

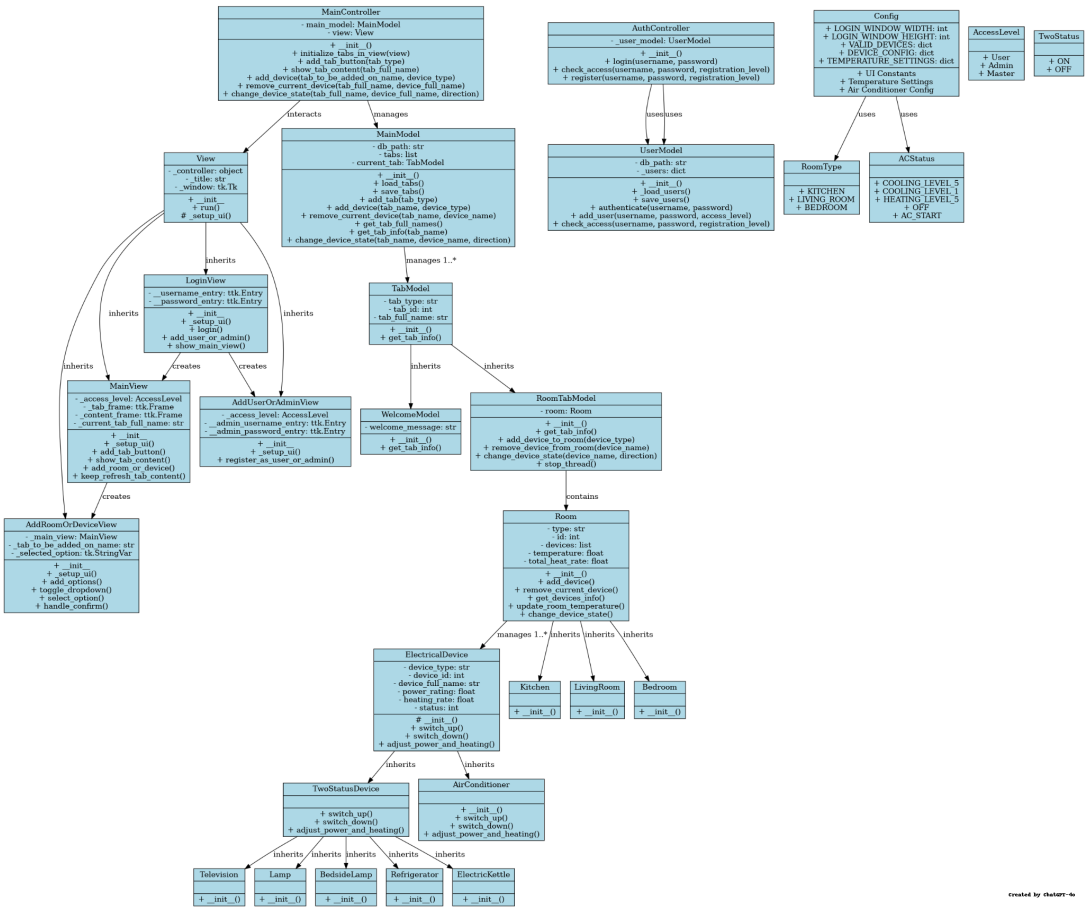
Despite these achievements, the project has limitations such as a simplistic GUI, unmanaged multithreading for scalability, and an open-loop temperature control system that lacks sensor feedback. Additionally, the project timeline was uneven, with most features implemented towards the end, limiting opportunities for testing and optimization.

Overall, IntelligentHome provides a solid foundation for smart home systems, with clear potential for future improvements. Enhancements such as a more robust GUI, sensor-driven closed-loop control, optimized threading, and generalization of classes would significantly improve scalability, performance, and usability in subsequent versions.

Acknowledgments

This project was supported in part by OpenAI's ChatGPT, which provided assistance in drafting the report, generating code snippets, and refining design ideas. The integration of ChatGPT significantly streamlined the development and documentation process.

7 Appendix

A UML Diagram

The UML diagram of the IntelligentHome system is presented below: